Au26 - GPUCompute
Homework 0 : Preliminaries and Refreshers

Learning Objectives

- Get familiar with using the AWS WorkSpaces *virtual desktop environment* (VDE)
- Get familiar with Visual Studio 2026 *integrated development environment* (IDE)
    - create and build a VS CUDA solution and project
    - use the debugger
        - set breakpoints and inspect variables and memory values
- Get familiar with key resources and references for Visual Studio development and the CUDA API.
- First look at a basic GPU Host application structure which uses the CUDA Runtime API and simple (GPU) Device kernel code for a vector addition
    - discussed in Lecture 1.
- Compile a CUDA application from the command line with NVCC
    - Experiment with CUDA compilation options / flags
    - Use CUDA binary tools to inspect and manage compiled artifacts
- Refresher on basic C programming, especially pointers and arrays

Development Resources

- Visual Studio 2026 - Community Edition (>= Sept 2026 update 18.10.2)
    - https://visualstudio.microsoft.com/vs/community/
    - [C++] https://learn.microsoft.com/en-us/cpp/overview/visual-cpp-in-visual-studio?view=msvc-170
- CUDA Toolkit (>= 13.4.2)
    - https://docs.nvidia.com/cuda/

Procedures

AWS WorkSpaces Applications VDE

1. The first time you access AWS WorkSpaces Applications VDE you will need to set up a new password - you should have received an email with your temporary password!
    a. Check your Trash or Junk/Spam folder as they often go there!

    b. Most likely **Subject:** `Start accessing your apps using Amazon AppStream 2.0`
        i. `from: no-reply`
2. You should also have received an email with your WorkSpaces VDE login information.
    a. **Subject:** `Amazon WorkSpaces Applications: New apps available`
        i. `from: no-reply`
    b. You will have a weekly VDE usage quota of 15 hrs / week maximum.
        i. after you reach this limit you'll need to wait until the next week
3. Log in to the link provided in the WorkSpaces email
    a. Select the Desktop option during the login process
    b. It may take a few minutes while session is prepared and instance spins up
4. After you log in to the Windows desktop you'll receive a welcome message popup with your current weekly time usage.
    a. IMPORTANT: the maximum duration of a single WorkSpaces session is 4 hours. At the end of the session the VDE instance will terminate and all data on it will be lost, so it is essential to save your work and any data you want to preserve to an off-instance persistent storage location (for example Google Drive or Microsoft OneDrive) before the session terminates!
    b. If your remaining time for the weekly quota is less than the 4 hour session maximum duration, your session will terminate sooner. You will receive several pop-up reminder messages before the session ends (at 10, 5, and 2 minutes) to remind you to save your data to an off-instance persistent storage location.
5. When you finish your WorkSpaces VDE session, be sure to save all work to a persistent off-VDE storage. Then click on your user name on the upper right toolbar and select Log Out. This will terminate your VDE session and free the resource for other users.

Visual Studio 2026 Usage

6. On the Windows toolbar at bottom, on the left side you'll have a Visual Studio shortcut icon (purple squiggly thing). Click it to open the Visual Studio 2026 IDE.
7. A startup window will open, select "Create a new project" option on the right..
8. In the Create a new project window, on the right change the "All languages" drop down to CUDA.
9. You should see the **CUDA 13.x Runtime** project option. Select it and click NEXT.
10. Choose a project name.
11. Note the Location path on your WorkSpaces system drive.
    a. Change the output directory to C:\Au26_GPU
    b. the new directory will automatically be created.
12. Uncheck the checkbox for "Place solution and project in the same directory"
    a. Name the Solution "Au26_GPUCompute"
    b. Name the Project "EX1_vecadd_cudart"
13. Click Create button
14. A VS solution and project will be created (look in the specified directory using File Explorer - available on the bottom Windows toolbar)

    a. The Project will be a subfolder of the Solution
        i. This will allow putting all your projects for the quarter into a single unified Solution for easy management.
    b. The main solution file has *.sln file extension.
    c. The main project file has a *.vcxproj file extension.
15. The Solution and Project will open in the VS IDE
    a. source code view for the kernel.cu file
        i. The CUDA project will contain a simple example of the vector addition Host and Kernel implementations, in a single source file named: kernel.cu
            1. *.cu is standard file extension for CUDA files.
        ii. Go through the entire kernel.cu file and understand its structure.
            1. Refer to the Lecture 1 slides
            2. It is using only the basic CUDA techniques we discussed in Lecture 1.
    b. Solution Explorer pane
        i. Understand the layout of the Solution Explorer pane.
        ii. As you add more projects to your solution, and the projects get more complex, Solution Explorer will be a very useful navigation tool.
        iii. TIP: if you right-click the project you can select many options, including Properties for the project. Project Properties will be used frequently to configure compilation and linking options.
16. From the top menu options, choose Build -> Build Solution.
17. In the bottom Output window the Build, compile and LINK stages should run and display outputs.
18. You should receive the following output

    ======= Build: 1 succeeded, 0 failed, 0 up-to-date, 0 skipped =======

Visual Studio Debugging

19. **Set breakpoints** in source code file kernel.cu by clicking in the source view pane on the leftmost grey column along edge. Set breakpoints on the following lines:
    a. breakpoints will show as red circles when enabled/set
    b. Line 11
    c. Line 17
    d. Line 29
    e. Line 46
    f. Line 66
20. Right click the breakpoint (red circle) on Line 17. You will see options to Delete or Disable the breakpoint. Click the Conditions… option.
    a. In the Conditions fields you can configure specific details for when a breakpoint will be triggered/hit.
    b. By default a breakpoint will always be hit.

    c. Close the breakpoint window.
21. Click the green arrow “Local Windows Debugger” in top toolbar to launch the Debugger.
22. Execution will stop at entry to the main( ) function, not at the first breakpoint. Why?
    a. Note the yellow arrow over the red breakpoint circle on left in Source view pane
    b. Also note the breakpoint on Line 11 inside the addKernel function has disappeared. Why?
23. Click the green arrow Continue button on top toolbar.
    a. Code will run up to the next breakpoint, Line 17.
24. Mouse over the arraySize variable
    a. A popup window will show that the value is uninitialized.
    b. Click F10 or the Step Over button on top VS toolbar. The yellow arrow will advance to Line 18.
    c. Now mouse over the arraySize variable again.
        i. Its value should have been assigned based on the code.
25. Using the top menu go to Debug -> Windows -> Memory -> Memory 1
    a. The Memory 1 window will appear in lower left window pane.
26. From top menu bar, select Debug -> Windows -> Immediate
    a. the Immediate Window will open in a lower right pane
    b. In the Immediate Window, type
        i. a, and press enter
        ii. the hexadecimal value of the memory location for the start of array a will be displayed, along with the first 5 array element values inside braces { … }
    c. In the Immediate Window, type
        i. &a[0], and press enter
        ii. Note that the value displayed is the same hexadecimal memory location as above for a.
        iii. For C arrays, the name of the array IS the memory address of the start of the array (the address of element 0).
27. In the Immediate Window, select the hexadecimal memory address value for a, and copy it. Paste it into the Address: field in the Memory 1 window. Press Enter.
    a. In the area below the Address: field, right click and select the following options
        i. 4-byte integer
        ii. Signed Display
    b. The view should update in the Memory 1 window. Note that the values displayed now agree with the values shown for the array a element values in the Immediate window.
28. Using the top VS toolbar, select Step Over or F10. Code execution will advance to the next line (19). Note that the values of the array a element memory locations in the Memory 1 window have updated to reflect the initialization values specified on Line 18.
29. In the Immediate Window, type
    a. a, and press enter
    b. Note that the array a values are now also updated in the Immediate window output.

30. Click green arrow Continue button
31. Note code execution will stop at the next breakpoint, inside the addWithCuda( ) function.
32. In the Immediate Window, type
    a. dev_a and press Enter
    b. type &dev_a and press Enter
33. Continue to the next breakpoint (Line 66)
    a. recheck values of dev_a and &dev_a in the Immediate window.
    b. Note that the value of &dev_a has not changed, but dev_a has. Why?
    c. Also note that the variable dev_a is a pointer variable type, defined on Line 46 as a pointer to an int (integer) type, int *dev_a = 0.
34. Continue to the next breakpoint (Line 29)
35. At the bottom Windows toolbar you'll see a purple VS debug console window icon.
    a. This is the VS debug output console window.
    b. Open it and note that it is empty.
36. Step over the printf( ) statement on line 29 to advance code to next line.
37. Now recheck the purple VS debug output window
    a. You should see the results of the addKernel execution displayed.
        i. The result should be correct if the GPU kernel executed correctly.
38. On the top VS toolbar, click the red square "Stop Debugging" button.
    a. The debug session will stop.
    b. The Immediate and Memory 1 debug windows will disappear and VS view will return to the source-code view.

CUDA Compilation with NVCC

39. In VDE desktop, on the bottom Windows toolbar click the black command prompt button
    a. A Visual Studio developer command prompt window will open
    b. Change directories to your Ex1_vecadd_cudart directory, which contains the kernel.cu CUDA source code file.
    c. At the command prompt type the following

```
nvcc -o vecadd kernel.cu -run
```

    d. The NVCC compiler (used to compile cuda c++ programs) will run, and then you'll see the same output from the vector addition program as above in the VS Debug console output.
    e. At the command prompt type
        i. `vecadd`
    f. You'll see the same vector addition output.
        i. NVCC compiled the *.cu source code into an application.
        ii. Use File Explorer to check the contents of the Ex1_vecadd_cudart directory.

1. You should see several new files
    a. vecadd (Application)
    b. vecadd.exp
    c. vecadd.lib

g. We will continue to discuss the NVCC compilation process/trajectory in greater depth later in the course.

First Look at CUDA Binary Tools

A CUDA program targets two processors: the CPU (host) and GPU (device), so nvcc separates and compiles the host and device portions for their respective architectures.

A .cubin is a GPU executable: an ELF file containing compiled NVIDIA GPU machine code (SASS) plus metadata/resource information for a specific architecture such as turing.

Normally, this GPU code is embedded inside the host executable, but it can also be generated as a standalone .cubin for inspection using cuobjdump or runtime loading through the CUDA Driver API.

Goals:

- >> `cuobjdump -lptx vecadd.exe`
    - list embedded PTX sections
- >> `cuobjdump -lelf vecadd.exe`
    - list embedded SASS sections
- (extract PTX and SASS code sections)
    - `cuobjdump --dump-ptx vecadd.exe`
    - `cuobjdump --dump-sass vecadd.exe`

Refresher: C Pointers and Arrays

40. We have already looked a bit at how C uses arrays and pointers in the prior part of the exercise, when you used the VS debugger to inspect the memory addresses and contents of C array and pointer variables in the kernel.cu source code.
41. Now you will modify the code in kernel.cu to use pointers instead of arrays for the Host-side arrays a, b, c defined on lines 18-20.
42. Comment out the current lines 18-20. Then immediately below them add the following new lines
    a. int* a = 0;
    b. int* b = 0;
    c. int* c = 0;
43. Save the file.

44. Click Build -> Rebuild Solution.
    a. The project should rebuild successfully. Why??
45. Go to the command prompt window and rebuild the code with NVCC from command line using the following
    a. `nvcc -o addvec kernel.cu`
    b. The code should compile correctly.
46. At the command line, run the addvec application.
    a. You will receive several errors when the application attempts to run. Why?
47. In the kernel.cu source code,modify the new lines we added in the Step above as follows
    a. int* a = (int*)malloc(arraySize * sizeof(int));
    b. int* b = (int*)malloc(arraySize * sizeof(int));
    c. int* c = (int*)malloc(arraySize * sizeof(int));
48. Save the file.
49. Go to the command prompt window and rebuild the code with NVCC from command line using the following
    a. `nvcc -o addvec kernel.cu`
    b. The code should compile correctly.
50. At the command line, run the addvec application.
    a. What are the results output now? Why??
51. In the kernel.cu source code, below the 3 lines added above, add the following code

```
for (int x = 0, *pa = a, *pb = b; x < arraySize; x++)
{
    *pa++ = x + 1;
    *pb++ = 10 * (x + 1);
}
```

52. Go to the command prompt window and rebuild the code with NVCC from command line using the following
    a. `nvcc -o addvec kernel.cu`
    b. The code should compile successfully. Ignore the Warnings for now.
53. At the command line, run the addvec application.
    a. What are the results output now? Why??
54. Now go to the result output printf( ) lines in the main( ) routine in kernel.cu, which follow the call to addWithCuda( ). Replace them with the following lines:

```
printf("{%d,%d,%d,%d,%d} + {%d,%d,%d,%d,%d} = {%d,%d,%d,%d,%d}\n",
    a[0], a[1], a[2], a[3], a[4],
    b[0], b[1], b[2], b[3], b[4],
    c[0], c[1], c[2], c[3], c[4]);
```

55. Go to the command prompt window and rebuild the code with NVCC from command line using the following
    a. `nvcc -o addvec kernel.cu`
    b. The code should compile successfully. Ignore the Warnings for now.
56. At the command line, run the addvec application.
    a. What are the results output now? Why??

57. In VS you can also use Build->Rebuild Solution and then rerun the code in the Local Windows Debugger.
    a. You should view the same results in the VS debug output console.

58. Be sure you understand each step taken in the above C Pointers and Arrays refresher section. Particularly understand the syntaxes used and why the pointer variables can be used to replace the array variables as shown. If you are confused or have any questions please ask for clarification from the TA or Professor.

CUDA Host Runtime API : Device Query

This section introduces using the CUDA Runtime API from a host C application to initialize host-side CUDA and use Runtime API functions to query the capabilities of current CUDA devices attached to the system. You will write a simple host application which will statically link to the Runtime API.

REF: https://docs.nvidia.com/cuda/cuda-runtime-api/cuda_runtime_api/apis.html

Starter Code Snippet

```
#include "cuda_runtime.h"
#include <stdio.h>

int main()
{
   // Initialize CUDA

   // Get number of CUDA devices

   // Loop through each device
   for (int device = 0; device < deviceCount; device++)
   {
      // Select device

      // Get driver/runtime versions

      // Get device properties

      // Print required device properties
   }
```

```
    return 0;
}
```

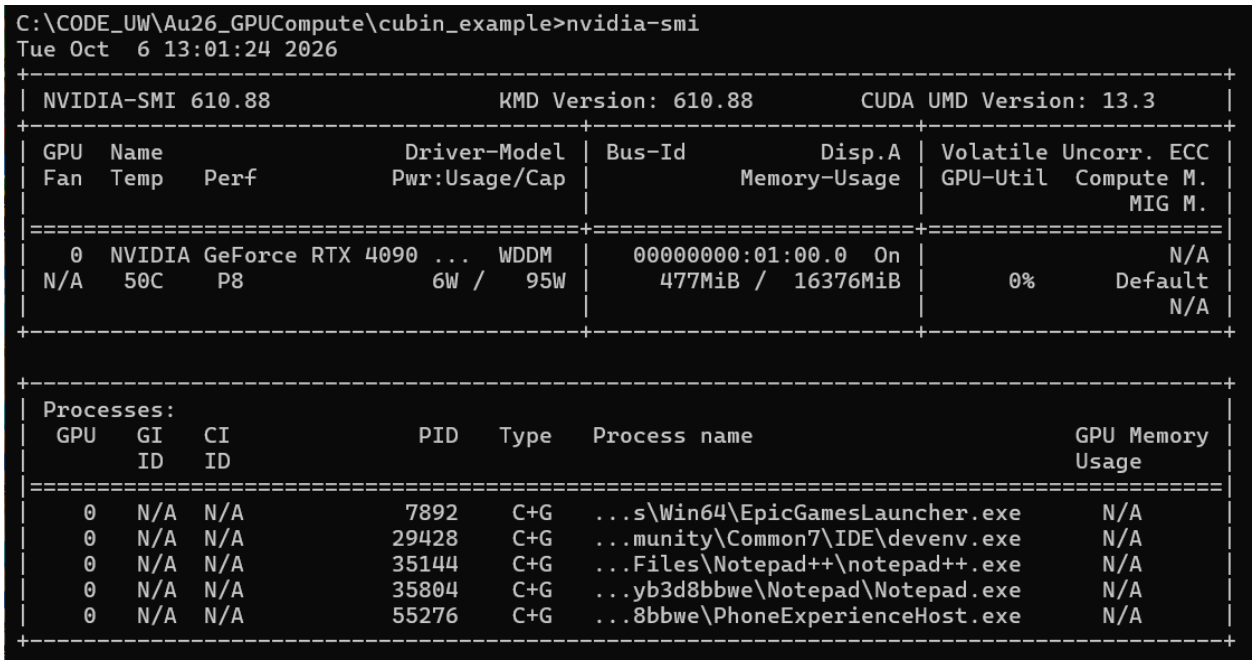
Specific CUDA Runtime API functions and properties to retrieve:

```
cudaInitDevice()
cudaGetDeviceCount()
loop for each device
        cudaSetDevice()
        cudaDriverGetVersion()
        cudaRuntimeGetVersion()
        cudaGetDeviceProperties()
                enumerate and output device properties:
                        l2CacheSize
                        major, minor compute capability
                        maxBlocksPerMultiProcessor
                        maxGridSize[3]
                        maxThreadDim[3]
                        maxThreadsPerBlock
                        maxThreadsPerMultiProcessor
                        multiProcessorCount
                        name[256]
                        regsPerBlock
                        regsPerMultiprocessor
                        sharedMemPerMultiprocessor
                        totalGlobalMem
                        warpsize
```

NVIDIA-System Management Interface Tool basics

NVIDIA also provides a binary tool (`nvidia-smi`) which can provide details on GPU type, usage, power, temperature, and lots of other information. Good to be aware of this tool! We'll do a few basic requests to get familiar. There is lots more.

Simplest usage:
At the x64 developer command prompt, type
>> nvidia-smi
It will show a display such as this:



From this display you can see what your system recognizes as the active NVIDIA display driver version, the installed CUDA version, the specific GPU device(s) present, and more.

Here's the NVIDIA-SMI docs (not pretty but useful!)

- https://docs.nvidia.com/deploy/nvidia-smi/index.html